

Linear Regression

self.eta $\rightarrow$ learning rate

self.n_iters $\rightarrow$ epochs.

self.weights } $\rightarrow$ these things we have to compute
self.bias } $\rightarrow$ set to None initially.

Fit $\Rightarrow$ Model Training. $\rightarrow$ #samples

n_samples, n_features = X.shape $\rightarrow$ (rows, cols)
self.weights = np.zeros(n_features) $\rightarrow$ #features.
self.bias = 0

for _ in range(self.n_iters): $\rightarrow$ epochs.
"""
Iterate n_iters times to optimize weights and bias.
y_pred is the predicted value, dw and db are gradients for weights and bias.
"""
y_pred = np.dot(X, self.weights) + self.bias

dw = (1 / n_samples) * np.dot(X.T, (y_pred - y))
db = (1 / n_samples) * np.sum(y_pred - y)

self.weights -= self.eta * dw
self.bias -= self.eta * db

initialize weights as '0'
and the size of the
weights array =
features.

Suppose $\rightarrow$

$$y = m_1x_1 + m_2x_2 + c$$

2 features.

$$\therefore W = [0 \ 0]$$

$\downarrow$
initialize 2 feats
with zero.

in every epoch. $\rightarrow$

- ① Make prediction.
- ② calculate derivatives (dw, db)
- ③ update weights & biases.

① Make prediction.

let, $\bar{X}$ = input features = $\begin{bmatrix} x_{11} & x_{12} \\ x_{21} & x_{22} \\ x_{31} & x_{32} \end{bmatrix}$

Shape.

$$X \in \mathbb{R}^{(3,2)}$$

x_1	x_2	y
x_{11}	x_{12}	y_1
x_{21}	x_{22}	y_2
x_{31}	x_{32}	y_3

input
(X)

$$W = \text{initialized weights.} \\ = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \rightarrow W \in \mathbb{R}^{(2,1)}$$

$b = \text{bias initialization} \rightarrow b = 0, b \in \mathbb{R}^{(1)}$

Prediction

$$\hat{y} = Xw + b$$

$$\begin{array}{ccc} \downarrow & \downarrow & \downarrow \\ x & w & b \\ (3,2) & (2,1) & (1,) \end{array}$$

$$= (3,1) + (1,) \therefore \hat{y} \in \mathbb{R}^{(3,1)}$$

$$y = wx + c$$

Eg:

$$\hat{y} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \rightarrow (3,1)$$

Compute Error

$$E = \hat{y} - y$$

$$E = (m,1) - (m,1) = (m,1)$$

Weight derivative (∂w)

$$\hat{y} = Xw + b$$

Cost Fn (MSE):

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

Matrix Form:

$$J = \frac{1}{2m} (\hat{y} - y)^T (\hat{y} - y)$$

$$\text{let, } e = \hat{y} - y = (wX + b) - y$$

$$\therefore J = \frac{1}{2m} e^T e$$

Now, find

$$\frac{\partial J}{\partial w}$$

$$J = \frac{1}{2m} (Xw + b - y)^T (Xw + b - y)$$

$$J = \frac{1}{2m} \sum (Xw + b - y)^2$$

$$= \frac{1}{2m} \sum (y - wX - b)^2$$

$$\frac{\partial J}{\partial w} = \frac{2}{2m} \sum (y - wX - b) \frac{\partial}{\partial w} (y - wX - b)$$

$$= \frac{1}{m} \sum (y - \hat{y}) (-x)$$

$$\therefore \frac{\partial J}{\partial w} = \frac{1}{m} (\hat{y} - y)(x)$$

$\downarrow \quad \quad \downarrow$
 $(m,1) \quad (m,n)$

$$\therefore \frac{\partial J}{\partial w} = \frac{1}{m} X^T (\hat{y} - y)$$

$\uparrow$
 $(n,m) \cdot (m,1) = (n,1)$

bias derivative. (∂b).

$$\begin{aligned}\frac{\partial L}{\partial b} &= \frac{2}{2n} \sum (\gamma - wx - b) \frac{\partial}{\partial b} \sum (\gamma - wx - \underline{b}) \\ &= \frac{1}{n} \cdot \sum (\gamma - wx - b) (-1) = -\frac{1}{n} \sum (\gamma - \hat{y})\end{aligned}$$

$$\therefore \boxed{\frac{\partial L}{\partial b} = -\frac{1}{n} \sum (\gamma - \hat{y})}$$

Now. Apply Gradient descent to update w & b .

$$\boxed{w = w - \eta \cdot \frac{\partial J}{\partial w}}$$

$$\boxed{b = b - \eta \cdot \frac{\partial J}{\partial b}}$$

η / α /
learning rate.

Predict

in the time of prediction.

given a new test data $X \rightarrow$ shape can be (m, n)

Suppose our trained model -

$$w = \begin{bmatrix} 4 \\ 2 \end{bmatrix} \quad b = 1$$

$(2, 1)$ $(1,)$

data points.

fixed
(# features)

Mean, the eq. $\hat{y} = \overset{1}{\uparrow} w_1 x_1 + \overset{2}{\uparrow} w_2 x_2 + \overset{1}{\uparrow} b$

Matrix form, $\hat{y} = WX + b$

$\downarrow$ $\downarrow$
 $\begin{bmatrix} 4 \\ 2 \end{bmatrix}$ 1

Now, we suppose get test data $\rightarrow X = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$ (2,2)
 Means, 1st test data $\rightarrow x_1 = 1, x_2 = 2$

2nd test data, $\rightarrow x_1 = 2, x_2 = 3$

so, $\left. \begin{aligned} \hat{y}_1 &= 4 \times 1 + 2 \times 2 + 1 = 9 \\ \hat{y}_2 &= 4 \times 2 + 2 \times 3 + 1 = 15 \end{aligned} \right\}$

Matrix Form

$$\hat{y} = XW + b = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 2 \end{bmatrix} + 1$$

$$= \begin{bmatrix} 8 \\ 14 \end{bmatrix} + 1 = \begin{bmatrix} 8 \\ 14 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 9 \\ 15 \end{bmatrix}$$

②

(Ans)

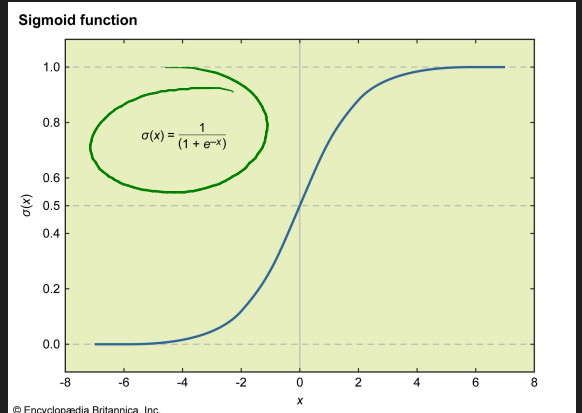
Logistic Regression

$\rightarrow$ same as Linear Regression. Only 1 extra step $\rightarrow$ Sigmoid fn

```
def sigmoid(x):
    x = np.clip(x, -500, 500)
    return 1 / (1 + np.exp(-x))
```

$\rightarrow$ to prevent
numeric
overflow.

$$\sigma(x) = \left(\frac{1}{1 + e^{-x}} \right)$$



```
def __init__(self, eta=0.001, epochs=1000):
    self.eta = eta
    self.epochs = epochs
    self.weights = None
    self.bias = None
    self.X_mean = None
    self.X_std = None
    self.loss_history = []
```

$\rightarrow$ eta $\rightarrow$ alpha / learning rate.

$\rightarrow$ initialize as none.

$\rightarrow$ to store loss.

```
def fit(self, X, y):
    n_samples, n_features = X.shape
    self.weights = np.zeros(n_features)
    self.bias = 0
    # Standardize X once at the start
    self.X_mean = np.mean(X, axis=0)
    self.X_std = np.std(X, axis=0)
    self.X_std = np.where(self.X_std == 0, 1e-10, self.X_std) # Avoid division by zero
    X_normalized = (X - self.X_mean) / self.X_std
    self.loss_history = []

    for epoch in range(self.epochs):
        linear_pred = np.dot(X_normalized, self.weights) + self.bias
        predictions = sigmoid(linear_pred)

        # Compute loss (binary cross-entropy)
        loss = -np.mean(y * np.log(predictions + 1e-15) + (1 - y) * np.log(1 - predictions + 1e-15))
        self.loss_history.append(loss)
        dw = (1 / n_samples) * np.dot(X_normalized.T, (predictions - y))
        db = (1 / n_samples) * np.sum(predictions - y)

        self.weights -= self.eta * dw
        self.bias -= self.eta * db

    # Print progress every 100 epochs
    if epoch % 100 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.4f}, Bias: {self.bias:.4f}")
```

Fit Steps

- ① Extract #samples, #features
- ② initialize weights with 0
- ③ bias with 0
- ④ Normalize X.

loop - epoch

- ① linear prediction
- ② apply sigmoid fn.
- ③ calculate loss & store.
- ④ calculate $\frac{dJ}{dw}, \frac{dJ}{db}$
- ⑤ update w & b

① Linear Prediction.

$$Z = Xw + b \rightarrow (m,)$$

$\swarrow$ $\downarrow$ $\searrow$
 (m, n) $(n, 1)$ $(1,)$

Eg.

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix} \quad w = \begin{bmatrix} 0.5 \\ 0.2 \end{bmatrix} \quad b = 0.1$$

$\begin{matrix} (2,2) \\ \uparrow \\ (2,1) \\ \uparrow \\ (1,) \end{matrix}$

$$\therefore Z = \begin{bmatrix} 1(0.5) + 2(0.2) + 0.1 \\ 2(0.5) + 3(0.2) + 0.1 \end{bmatrix} = \begin{bmatrix} 1.0 \\ 1.7 \end{bmatrix}$$

② Apply Sigmoid

$$\hat{y} = \sigma(z)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\hat{y} = \begin{bmatrix} \sigma(1.0) \\ \sigma(1.7) \end{bmatrix} = \begin{bmatrix} 0.731 \\ 0.845 \end{bmatrix}$$

③ Calculate Loss

BCE $\Rightarrow$
Binary CrossEntropy

$$L = -\frac{1}{m} \sum [\gamma \log \hat{\gamma} + (1 - \gamma) \log (1 - \hat{\gamma})]$$

④ Calculate derivatives

$$w, b \rightarrow z \rightarrow \hat{y} \rightarrow L$$

Predict

```
def predict(self, X):
    linear_pred = np.dot(X, self.weights) + self.bias
    y_pred = sigmoid(linear_pred)
    class_pred = [0 if y <= 0.5 else 1 for y in y_pred]
    return class_pred
```

- ① calculate linear prediction
- ② apply sigmoid.
- ③ apply threshold to get classes.

③ **KNN** :

```
def euclidean_distance(self, a, b):
    return np.sqrt(np.sum((a - b) ** 2))

def manhattan_distance(self, a, b):
    return np.sum(np.abs(a-b))

def fit(self, X_train, y_train):
    self.X_train = X_train
    self.y_train = y_train.to_numpy()
```

$$d_e = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

$$d_m = \sum_{i=1}^n |a_i - b_i|$$

} in fit just store the data — no training

```
def predict(self, X_test):
    y_pred = []

    for x in X_test:
        distances = []

        for i in range(len(self.X_train)):
            if self.dist == 'manhattan':
                d = self.manhattan_distance(x, self.X_train[i])
            else:
                d = self.euclidean_distance(x, self.X_train[i])
            distances.append((d, self.y_train[i]))

        distances.sort()
        k_nearest = distances[:self.k]

        labels = [label for _, label in k_nearest]

        counter = Counter(labels)
        prediction = counter.most_common(1)[0][0]

        y_pred.append(prediction)

    return np.array(y_pred)
```

Steps

- ① Take test sample
- ② calculate distance from the training datapoints.
- ③ sort distances & corresponding labels.
(All distances in ascending)
- ④ Select 1st k numbers.
- ⑤ Extract labels of k numbers
- ⑥ Majority voting.

training data

x1	x2	Label
1	2	A
2	3	A
5	6	B
6	7	B

$$X_{\text{train}} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \\ 5 & 6 \\ 6 & 7 \end{bmatrix}$$

$$Y_{\text{train}} = \text{label} = [A \ A \ B \ B]$$

Now →

① take test sample. $X = (3, 4)$, $K = 3$ (let)
dist = euclidean. }

② calculate distances.

$d_{(1,2)} = \sqrt{(3-1)^2 + (4-2)^2} = 2.83$ $d_{(2,3)} = \sqrt{(3-2)^2 + (4-3)^2} = 1.41$

list of X from (1,2) $d_{(5,6)} = 2.83$ $d_{(6,7)} = 4.24$

∴ distances = $\begin{bmatrix} (2.83, 'A'), \\ (1.41, 'A'), \\ (2.83, 'B'), \\ (4.24, 'B') \end{bmatrix}$

③ Sort.

$\begin{bmatrix} (1.41, A), \\ (2.83, A), \\ (2.83, B), \\ (4.24, B) \end{bmatrix}$

④ select 1st K. ($K=3$) here.

$\begin{bmatrix} (1.41, A), \\ (2.83, A), \\ (2.83, B) \end{bmatrix}$

⑤ Extract labels.

$[A, A, B]$

⑥ Majority Voting.

$\{ \textcircled{A:2} \rightarrow \textcircled{A} \checkmark, B:1 \}$

4

Naive Bayes

want to find

For a new sample we actually

$P(\text{Yes} | X) \times P(\text{No} | X)$

input features

Based on:

Bayes's Theorem

$$P(C | X) = \frac{P(X | C) \cdot P(C)}{P(X)}$$

class (Y/N)

input sample.

$$P(c|x) = \frac{P(x|c) \cdot P(c)}{P(x)}$$

$P(c|x)$: posterior probability.
 $P(x|c)$: likelihood.
 $P(c)$: class prior.
 $P(x)$: predictor prior.

Since $P(x)$ is the same for all classes we ignore that.

fit Steps

```
def __init__(self):
    self.features = None
```

```
self.p_yes = None
self.p_no = None
```

```
self.mean_yes = None
self.mean_no = None
```

```
self.std_yes = None
self.std_no = None
```

```
def fit(self, X, y):
    self.features = X.columns → ①
```

```
yes_data = X[y == 1]
no_data = X[y == 0] ] ②
```

```
# prior probabilities P(yes) and P(no)
self.p_yes = len(yes_data) / len(X)
self.p_no = len(no_data) / len(X) } ③
```

```
# mean and std for yes class
self.mean_yes = yes_data.mean()
self.std_yes = yes_data.std()
```

```
# mean and std for no class
self.mean_no = no_data.mean()
self.std_no = no_data.std()
```

} ④ & ⑤

① Store the feature names.

② Separate the dataset by class.

yes (y=1)
no (y=0)

③ Calculate prior probabilities.

$P(\text{yes}) = (\# \text{yes}) / \text{total}$
 $P(\text{no}) = (\# \text{no}) / \text{total}$

④ Calculate Mean of every feature for yes & no class.

⑤ Calculate std of every feature for yes & no

Predict

```
def gaussian_probability(self, x, mean, std):
    exponent = np.exp(
        -((x - mean) ** 2) / (2 * (std ** 2))
    )
    return (1 / (std * np.sqrt(2 * np.pi))) * exponent
```

```
# predict one sample
def predict_single(self, sample):
```

```
# start with prior probabilities
prob_yes = self.p_yes
prob_no = self.p_no ①
```

```
# calculate probability for yes
for feature in self.features:
    prob_yes *= self.gaussian_probability(sample[feature], self.mean_yes[feature], self.std_yes[feature])
```

```
# calculate probability for no
for feature in self.features:
    prob_no *= self.gaussian_probability(sample[feature], self.mean_no[feature], self.std_no[feature])
```

```
# return class with higher probability
return 1 if prob_yes > prob_no else 0 — ④, ⑤
```

```
def predict(self, X):
    predictions = []
    for _, row in X.iterrows():
        prediction = self.predict_single(row)
        predictions.append(prediction)
    return np.array(predictions)
```

Steps

① Start with Prior Probabilities

② For each feature calculate Gaussian Probability for yes and no class.

③ Compare Final Probability

④ Return Prediction.

① Start with prior probability. $\rightarrow \begin{cases} \text{Prob_yes} = \text{self.P_yes} \\ \text{Prob_no} = \text{self.P_no} \end{cases}$

② Calculate gaussian Probability $\rightarrow$ and Multiply with prior.

$$P(x|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$

$\downarrow$
Gaussian Probability.

Suppose

Height $\rightarrow P(172|\text{yes})$

Weight $\rightarrow P(68|\text{yes})$

$$\therefore \text{Prob}_{\text{yes}} = \underbrace{P(Y)}_{\text{(Prior Probability)}} \times P(\text{Height}|\text{yes}) \times P(\text{Weight}|\text{yes})$$

$$\text{Prob}_{\text{No}} = P(N) \times P(\text{Height}|\text{No}) \times P(\text{Weight}|\text{No})$$

③ Compare Final Probability.

Suppose $(\text{Prob}_{\text{yes}} > \text{Prob}_{\text{No}}) \rightarrow$ So. ④ Return yes.